

Mettre l'application CHRUTH en ligne

L'app se lance en local sans rien configurer. La mise en ligne sert à la consulter depuis un téléphone, sans PC allumé.

En local, tout de suite

Double-clic sur `LANCER_APP_CHRUTH.bat`, ou :

```
python -m streamlit run CHRUTH_APP.py
```

L'application s'ouvre sur la veille ; la page « Messages et CRM » est dans le menu de gauche.

Elle lit `etat/veille.json` dans le dossier du projet. Sans veille cloud encore en service, ce fichier se remplit avec :

```
python ao_maximilien_veille.py
```

En ligne, sur Streamlit Community Cloud

Cinq étapes. Les deux premières engagent ton identité : elles ne se délèguent pas.

1. Se connecter — à faire soi-même

<https://share.streamlit.io> → **Continue to sign-in** → GitHub.

Streamlit demande l'autorisation d'accéder à tes dépôts : tu acceptes ses conditions d'utilisation et tu lui accordes des permissions sur ton compte.

2. Créer un jeton GitHub pour l'app — à faire soi-même

L'app doit lire et réécrire `etat/veille.json` sur la branche `ao-state` d'un dépôt **privé**. Sans jeton, elle démarre mais reste vide.

GitHub → `Settings > Developer settings > Personal access tokens > Fine-grained tokens` → **Generate new token** :

Réglage	Valeur
Repository access	Only select repositories → <code>CHRUTH</code> , et lui seul
Contents	Read and write — lire et réécrire l'état
Actions	Read and write — le bouton « Mettre à jour maintenant »
Expiration	90 jours, à renouveler

N'utilise pas le jeton déjà présent dans ton gestionnaire d'identifiants Windows. Il porte les droits `repo`, `user`, `gist` sur **tous** tes dépôts. Le déposer chez un service tiers lui confierait bien plus que ce qu'il a besoin de faire. Un jeton limité à un dépôt et à deux permissions se révoque en un clic sans rien casser d'autre.

3. Déployer

`New app` → depuis un dépôt existant :

Champ	Valeur
Repository	<code><votre-compte>/CHRUTH</code>
Branch	<code>main</code>
Main file path	<code>CHRUTH_APP.py</code> — le point d'entrée, pas <code>app_veille.py</code>

Community Cloud installe `requirements.txt` puis démarre l'app. Un test de la suite (`test_dependances_app.py`) vérifie en continu que ce fichier couvre tout ce que l'application importe : c'est la panne de déploiement la plus courante.

4. Renseigner les secrets

`Settings > Secrets`, au format TOML. **Colle-les toi-même** : un jeton ne se dicte pas.

```
CHRUTH_VEILLE_SOURCE = "github"
CHRUTH_GITHUB_REPO = "<votre-compte>/CHRUTH"
CHRUTH_GITHUB_TOKEN = "<le jeton de l'etape 2>"
```

Sans `CHRUTH_GITHUB_TOKEN`, l'app est **vide et en lecture seule** : elle ne peut ni lire l'état du dépôt privé, ni enregistrer une correction.

Community Cloud expose chaque secret comme variable d'environnement : c'est ainsi que `reglages.py` et `llm_client.py` les lisent, sans code particulier.

Pour que la page **Messages et CRM** génère en ligne, ajouter une clé cloud — une seule suffit, `moteur_auto()` la trouve. Ollama ne tourne pas sur Community Cloud.

```
ANTHROPIC_API_KEY = "<ou MISTRAL_API_KEY, ou GROQ_API_KEY>"
```

5. Vérifier

Dix pages dans le menu de gauche : Accueil, Veille, Collecte, Base de données, Acheteurs de la semaine, Carte, Messages et CRM, Pilotage, Réglages et Développeur.

- L'application ouvre sur l'**Accueil** : échéances les plus proches et AO retenus par le tri. En ligne, la base locale n'existe pas — les chiffres qui en viennent restent donc à zéro, et c'est normal.
- Le fil affiche les AO, du plus récemment publié au plus ancien.
- La jauge **Score minimum** de la barre latérale filtre la liste.
- « Afficher les AO rejetés par le tri » les fait apparaître avec leur motif.
- Un clic sur **Pas pertinent** survit à un rechargement de la page.
- Le bouton **Mettre à jour maintenant** déclenche le workflow — il ne collecte pas depuis l'app, ce qui marquerait les AO comme vus sans les notifier.
- **Réglages** affiche les vrais destinataires. En changer un, puis vérifier au run local suivant que `ao_alertes.charger_destinataires()` les renvoie.
- Le mot de passe d'application n'apparaît **nullement** : il n'est pas dans l'état partagé, et la page Réglages ne le lit pas.

Ce qui ne fonctionne pas en ligne, et pourquoi

Community Cloud redémarre l'application quand il veut, avec un disque neuf. Tout ce qui n'est pas dans le dépôt ou dans l'état partagé disparaît.

Fonction	En ligne	Raison
Veille, tri, corrections	✓	dans <code>etat/veille.json</code> sur <code>ao-state</code>
Réglages	✓	même état partagé
Messages IA	✓ avec une clé cloud	Ollama est local
Pilotage	✗ affiche 0	les KPI viennent de la base SQLite locale, gitignorée : elle n'est pas publiée. La page le dit explicitement en ligne plutôt que de renvoyer vers une mise à jour qui ne la remplirait pas
Saisies CRM	⚠ éphémères	<code>crm/suivi_clients.csv</code> est sur le disque du conteneur, effacé au redémarrage

Le Pilotage et le CRM restent complets dans l'application lancée **sur le PC**, qui lit la vraie base. C'est le partage voulu : le téléphone sert à suivre la veille et à corriger le tri, le poste sert à travailler les données.

Qui pourra ouvrir le lien

Le dépôt est **privé**, donc l'app l'est aussi : un visiteur sans droits dessus doit s'authentifier — compte Google, ou lien à usage unique valable 15 minutes. Le lien seul ne suffit pas.

Conséquence : le **guide des messages** peut être activé en ligne, puisque l'accès est réellement restreint. Il reste masqué tant que `CHRUTH_VEILLE_GUIDE` n'est pas mis à `"1"` dans les secrets — à décider une fois la liste des personnes autorisées arrêtée.

Pour donner l'accès à quelqu'un : `Settings > Sharing` de l'app.

Comment l'app et la veille se partagent le fichier

Les deux écrivent `etat/veille.json` sur la branche `ao-state` : le workflow par `git`, l'app par l'API GitHub. L'app relit avant d'écrire et réessaie une fois en cas de conflit — un run de veille qui écrit entre-temps ne fait donc rien perdre.